

All Out War : Shogun

Project Overview

Description :

All Out War: Shogun is a real time strategy game that allows players to impersonate a warlord leading a powerful army to conquer the highest national position, the Shogun, the mighty ruler of Japan. The game allows players to choose a starting clan from five different clans, each with its own specialties. Players can upgrade their city hall to increase their army size.

All goods and resources come from citizens' taxes. If players cannot manage the city's finances properly, they may fall into debt. On the other hand, if players become too wealthy, they may overtax the city, which can lead to rebellion and the loss of entire cities. All Out War: Shogun tests players' strategic abilities in both warfare and financial management.

Problem the Project Solves :

In All Out War: Shogun, players must manage warfare and city finances, but the game ***lacks a system to clearly track and analyze player actions and resource usage***. As a result, players may not understand why they fall into debt, lose cities, or trigger rebellions.

This project aims to solve this problem by developing a system that ***records player actions and presents the data through graphs and reports***, helping players evaluate their strategies and make better decisions.

Target users :

The target users are historical strategy game players who are interested in realistic town management and warfare during the Sengoku period in Japan.

Key features :

- **Actions of conquest :**

→ **March** : march armies in the path within the map , the army size will dictate how far the army can go , if the army is too big , it can go less further. Player and enemy cannot march pass obstacles and castles that they don't belong to.

*** *MUST FOLLOW ROUTES* ***

→ **Attack** (*during march*) : player or enemy can attack each other's army if they intercept each other along the routes or by choice.

→ **Ambush** (*during march*) : player or enemy can ambush in the forest on the map . They must be in the bush or forest. Once ambushed , the opponent will be forced to fight in that turn and cannot make any choice.

→ **Siege** : player or enemy can siege the city , the total defender can be recruited soldiers + default guards or just default guards if there are no recruited soldiers in the city.

- **Citizens/Tax : Main Income**

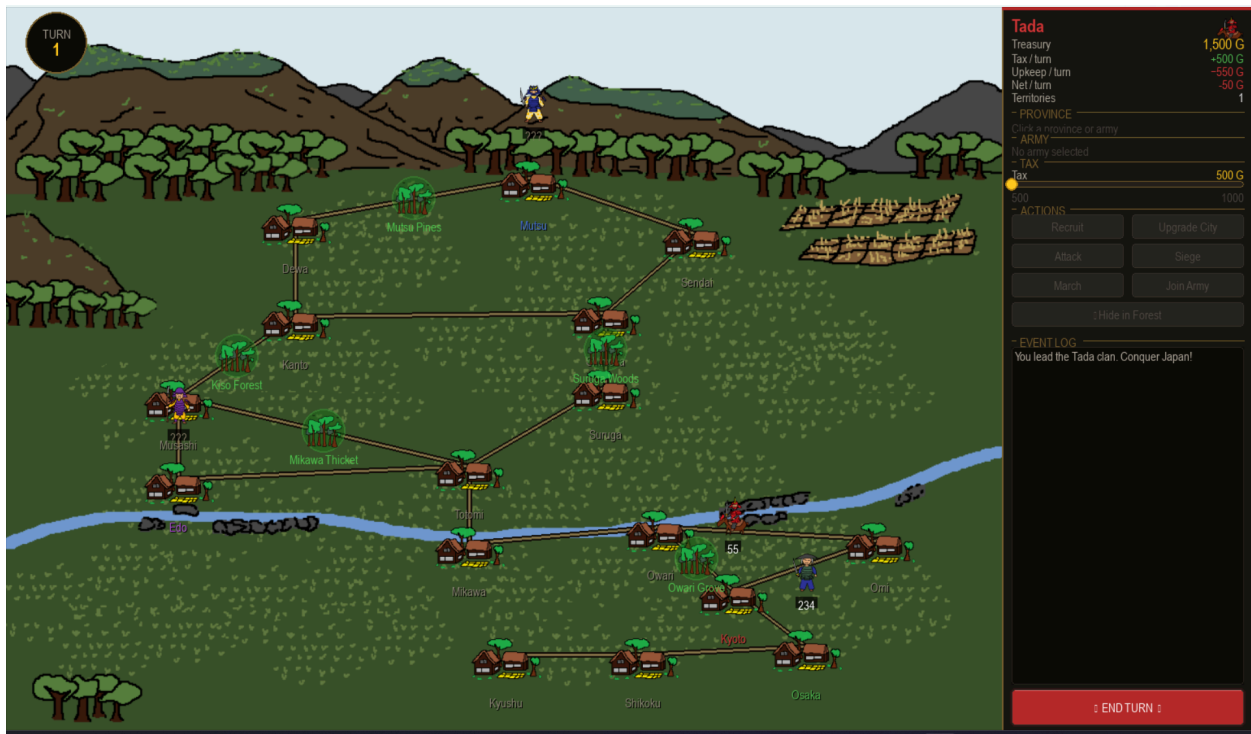
Consist of rage levels : 1 - 6 based on how players tax the city and the level of the city. Collecting tax scales from 500-1000 * multiplier of city level

→ If $(\text{garrisons unit} \times 10) + (\text{soldier unit} \times 10) > \text{tax}$, rage level = default as 3 when collecting.

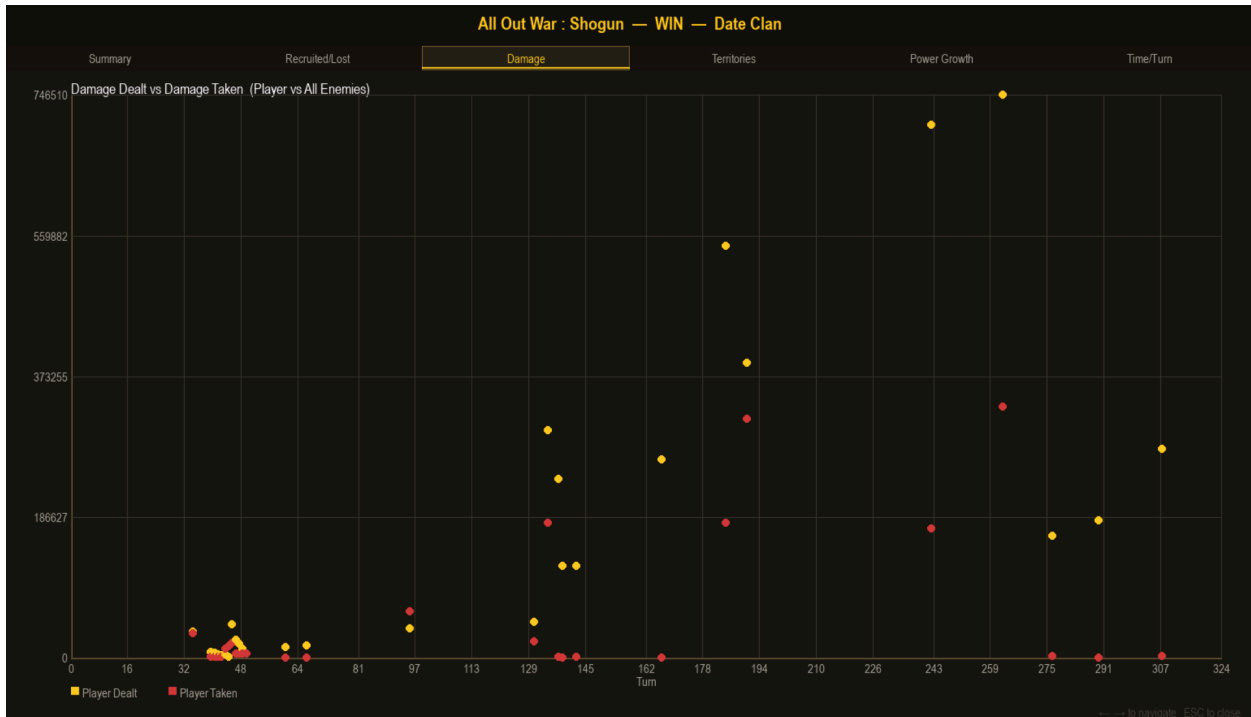
→ Else If $(\text{garrisons unit} \times 10) + (\text{soldier unit} \times 10) < \text{tax}$, rage level will be -1 for next turn

→ Else , rage level will be adding +1 every turn

GAMEPLAY :



DATA VISUALIZATION:



Project Proposal :

<https://docs.google.com/document/d/1jU-tGuk2UVzrdr7KiE2AUEegWUfTO7R71r7vL6LVxic/edit?usp=sharing>

YouTube presentation : <https://youtu.be/6YiqYjAkW8I>

Concept

Background (Inspiration):

The inspiration for this game comes from Total War: Shogun 2, a large-scale real time strategy game developed by Sega. I found the game to be rich in mechanics and interesting strategic systems. All Out War: Shogun will be a smaller 2D game inspired by this reference, mainly based on historical actions and events.

Highlight of the game :

All Out War : Shogun heavily relies on ***real time strategy gameplay, financial management, and strategic decision-making in each turn.***

The goal of the game is to conquer Japan and become the Shogun.

However, this will not be easy because there are many obstacles along the way, similar to the historical events of Japan during the Sengoku period (the Warring States period).

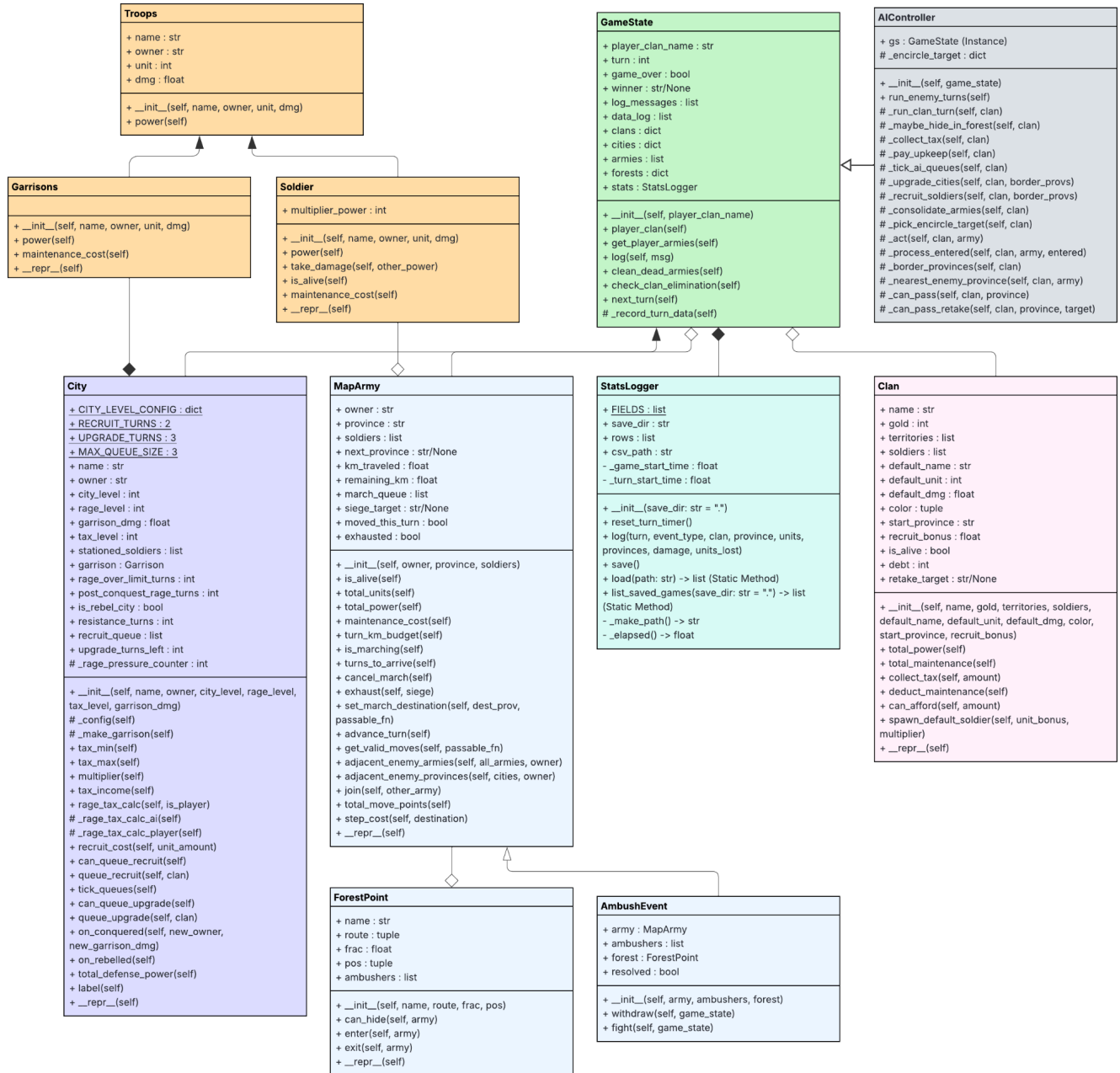
Objectives :

The main objective of the game is for the player to conquer all 16 provinces on the map while overcoming enemy bots that act as major obstacles throughout the campaign. Players must strategically manage their resources, armies, and territories in order to win the game.

Condition to win : Conquer all city of the Japan

Condition to lost : Player territory reach 0

UML Class Diagram :



Object-Oriented Programming Implementation

Clan's Soldier default info

- Tada Clan - ("Katana Cavalry", "Tada", 55 , 10.5, 1)
- Date Clan - ("Odachi Senshi", "Date", 60 ,10, 1)
- Nori Clan - ("Katana Ashigaru", "Nori", 180 , 2.9, 1)
- Abe Clan - ("Yari Senshi", "Abe", 100 , 6.2, 1)

Class : AIController

- **Economic Management:** Handles tax collection (adjusting levels based on debt), pays unit upkeep, and applies periodic "difficulty bonuses" to keep the AI competitive.
- **Infrastructure & Logistics:** Decides which cities to upgrade and where to recruit new units, prioritizing border provinces.
- **Strategic Targeting:** Reassesses the "best" target every turn (prioritizing the player and lost territories) and manages army movement/marching.
- **Combat Execution:** Triggers automated field battles and sieges based on calculated winning probabilities.

Class : City

Helpers function : make_city()

A convenience factory function used to instantiate a new City object with standardized default values.

- **Infrastructure Management:** Tracks the current city level (from Village to Citadel) and manages the 3-turn upgrade process.
- **Economic System:** Calculates tax income based on a slider value and a city-level multiplier.

- **Public Order (Rage):** Implements complex "Rage" mechanics that differ between players and AI. It tracks unrest caused by high taxes, passive drift, and recent conquest, potentially leading to armed rebellions.
- **Production Queues:** Manages the recruitment of new Soldier units and prevents simultaneous upgrading and recruiting.
- **Defense:** Calculates total defense power by combining the permanent Garrison with any stationed_soldiers currently in the city.

Production Queue:

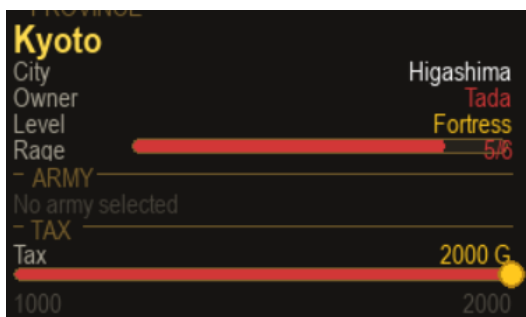


Recruiting soldiers



Upgrading City

Economics and Public Order :



Rage and Taxing



Taxing and Net income

Class : Clan

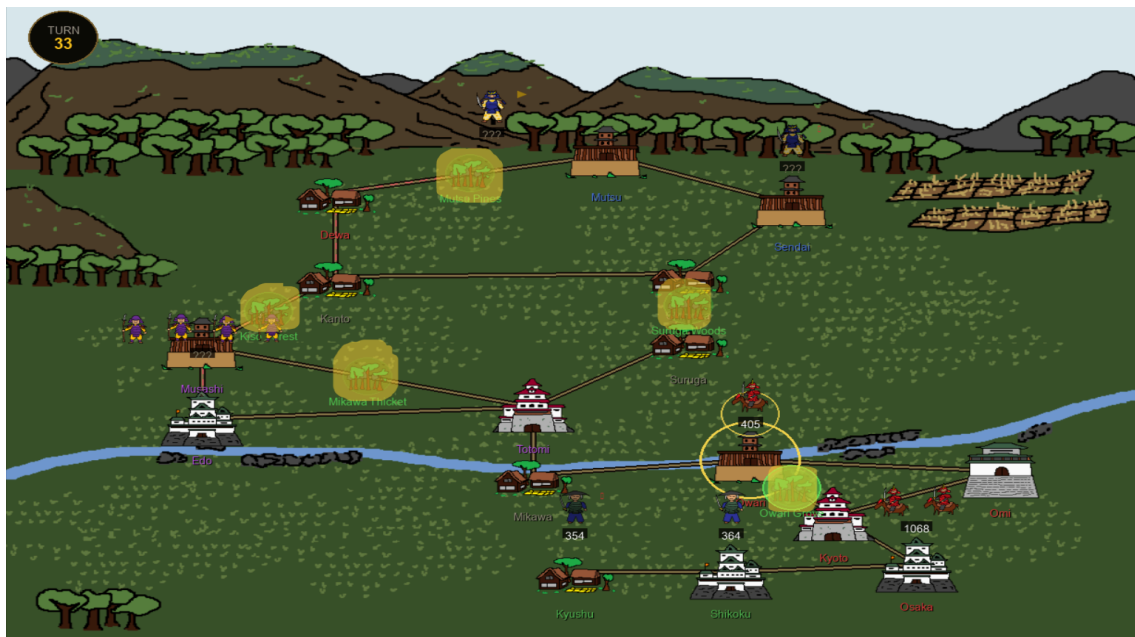
Helpers function : *make_clans()*

factory function that initializes the *four* primary factions with distinct starting attributes:

1. **Tada** : Elite cavalry focus with high base damage (10.5).
 2. **Date** : Odachi specialists with high base damage (10.0).
 3. **Nori** : Focuses on overwhelming numbers; starts with 180 units and a 1.30x recruitment bonus.
 4. **Abe** : Yari (spear) formation specialists with balanced damage (6.2) and a strong bonus against cavalry.
-
- **Resource Management:** Tracks current gold reserves and accumulated debt if upkeep exceeds available funds.
 - **Military Identity:** Stores default unit types, base damage stats, and a recruit_bonus which can increase the number of soldiers produced per recruitment order.
 - **Strategic State:** Tracks whether the clan is still alive and identifies specific "retake targets" if they lose a home province.
 - **Soldier Production:** Includes methods to spawn soldiers based on the clan's specific stats and current bonuses.

Class : ForestPoint

- **Location:** Forests are positioned on the routes between two specific provinces.
- **State Management:** Tracks which armies are currently hiding inside.
- **Visibility:** When an army "enters" a forest, it is marked as hidden and its map position snaps to the forest's coordinates.
- **Auto-Exit:** Armies are automatically revealed if they begin marching or move away from the provinces connected to the forest route.



ForestPoints (Highlighted)

Class : AmbushEvent

This class handles the logic triggered when a marching army (the victim) enters a route containing hidden enemy forces. It offers the victim two choices for resolution:

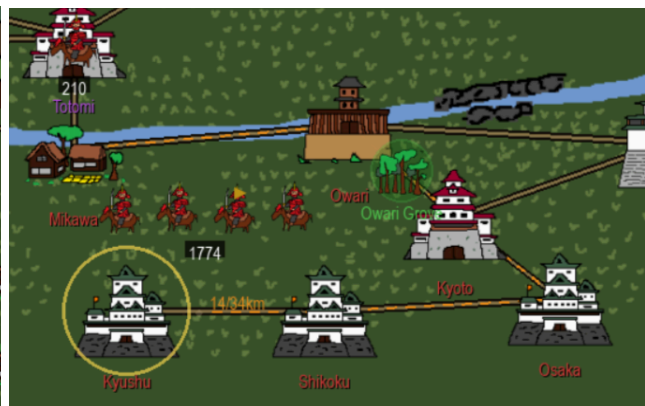
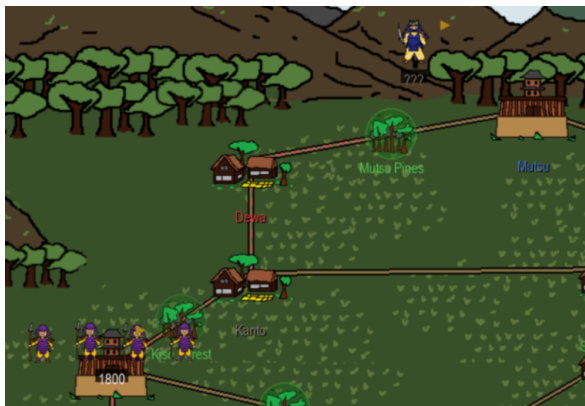
- **Withdraw:** The victim retreats to their origin province and loses all remaining movement for the turn.
- **Fight:** A battle is initiated where the ambushers receive a "first-strike" bonus before normal combat resolution. Ambushing armies are revealed to the map after the fight.

Class : GameState

- **Initialization:** It constructs the world by initializing all clans, assigning a city to every province on the map, and spawning the starting military forces for each faction.
- **City & Province Management:** It maps provinces to their starting owners and ensures that city garrisons use the specific damage stats of their owning clan.
- **Turn Logic:** It handles the transition between turns, including triggering AI logic for non-player clans and processing per-turn logistics.
- **Event Logging:** Maintains a visual log_messages list for the UI and a data_log for statistical tracking and CSV exports.
- **Condition Checking:** Periodically checks for clan elimination and determines if a winner has been decided.
- **Warning System:** If a city's "Rage" level is too high, it logs a warning for the owning player, indicating how many turns remain before a revolt.
- **Rebellion Execution:** When a rebellion triggers, it converts the city to the "Rebels" faction, clears existing siege targets, and logs the loss of the territory.
- **Army Tracking:** Manages a global list of *MapArmy* objects, ensuring they are correctly positioned and updated.
- **Ambush Detection:** Utilizes the check_ambush utility to see if any movement on the map triggers an interaction with hidden units.
- **Data Captured:** It logs the total military power and number of territories for every active clan.
- **Performance Metrics:** It tracks player-specific data such as gold reserves and total soldier counts for analytics.

Class: MapArmy

- **Position Tracking:** It tracks whether an army is stationed at a specific province or is "mid-route" between two nodes using a kilometer-based coordinate system.
- **Movement Logic:** It calculates travel progress based on the army's size (power), the terrain type (e.g., mountains), and specific clan bonuses (e.g., the Tada clan's speed boost).
- **Pathfinding:** It uses a Breadth-First Search (**BFS**) algorithm to set long-distance march queues and manages the step-by-step progress through those queues each turn.
- **Combat Readiness:** It tracks "exhaustion" states, which temporarily freeze an army's movement after they have successfully captured a province or completed a siege.
- **Stealth:** It manages the "hidden" status, allowing armies to become invisible to enemies when stationed in forest terrain.
- **Rendering Data:** It provides the *screen_pos()* method, which calculates the exact (x,y) pixel coordinates needed to draw the army on a 960×720 canvas, including smooth interpolation while moving between provinces.



Class : Troops

- **Attributes:** Stores the name, owner, unit count, and base damage per unit.
- **Power Calculation:** Provides a basic **power()** method calculated as $\text{Units} \times \text{Damage}$.

Class : Soldier(Troops)

- **Multiplier Power:** Features a ***multiplier_power*** attribute (defaulting to 1) to allow for temporary buffs or debuffs to combat effectiveness.
- **Combat Logic:** Includes a ***take_damage()*** method that reduces the unit count based on the incoming power from an opponent.
- **Maintenance:** Calculates a ***maintenance_cost()*** based on 10 gold (or resources) per unit.
- **Life Status:** Uses ***is_alive()*** to check if the unit count is greater than zero.

Class : Garrisons(Troops)

- **Simplified Logic:** It lacks the damage-taking and multiplier logic of the Soldier class, focusing instead on a ***static power()*** value and ***maintenance_cost()***.

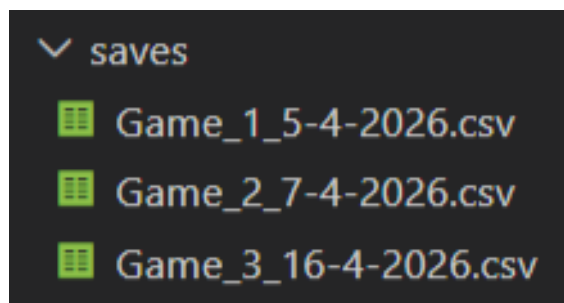
Feature	Soldier(Troops)	Garrison(Troops)
Movement	Mobile (Field Armies)	Stationary (Defenders)

Power Formula	Unit x times Dmg x times Multiplier	Unit x times Dmg
Cost	10 per unit	10 per unit
Health	Can die	Static (in this file)

Troops summaries table

Class : StatsLogger

- **Automatic CSV Creation:** Generates a uniquely named CSV file for every game session (e.g., Game_1_16-4-2026.csv) to ensure records are preserved.
- **Event Logging:** Captures specific game events like RECRUIT, BATTLE_WIN, SIEGE_WIN, and REBELLION.
- **Timer System:** Tracks time_elapsed for each turn to analyze how long players take to make decisions.



CSV creation

Data Feature

- Every player action is logged as its own row. One 20-turn game produces roughly **100–200** rows, eg: each battle logs 2 rows (one per side), each recruit logs 1 row. These will be collected in “**Raw data**” as a base data to generate **5 graphs** and **1 table**.
- All **5 graph** features are derived by filtering the event_type column , no data is duplicated.

- The game allows players to select and load statistics from previously completed games. Each game is stored as a separate CSV file and can be loaded to regenerate **graphs** and a **table** for analysis.

Example of Raw data structure :

turn	time	event_type	clan	province	units	provinces
3	12.4	RECRUIT	Tada	Kyoto	55	2
3	15.2	BATTLE	Tada	Musashi	0	3

Meaning : Tada has recruited for 55 units and also got into a battle that lost 55 units in turn 3. Tada gained final(3) - initial(2) = 1 province in turn 3.

Display in depth after the game ends (win or lost) for players. Graphs will be plotted with graph plotters.

Features	Objective	Method	Variable	Display
Soldiers Recruited vs Soldiers Lost	To see how efficient players can manage recruiting soldiers and determine hardness of Bots	Log 1 row per RECRUIT event and per BATTLE event in CSV	game.py	Stacked Bar Chart
Damage Dealt vs Damage Taken	To see total damage taken for player vs AI in term of attacking	Log 1 row per BATTLE/SIEGE/AMBUSH for each side	game.py	Scatter Plot
Cities Gained vs Cities Lost	To see how efficient player can play and determine how hard the BOT is in term of town	Log 1 row per SIEGE_WIN/SIEGE_LOSS event	game.py	Bar Chart

Displays overall military growth across turns.	To see how efficient player can recruit in term of gold , rage , AI enemies	Log 1 row per clan per turn (turn-end snapshot)	clans.py	Cumulative Line Chart
Time per turn	To collect average playtime for each game	Log 1 row per turn with elapsed seconds	game.py	Line Chart

Data Recording Method

The data will be recorded to a CSV file for each game exclusively. The data will be saved in format of ***game[#]_date/month/year*** , once restart the game again, a new CSV file will be created specifically for that new game.

Example: Game_1_29-3-2026.csv for Game_1 ,
Game_2_30-3-2026.csv for Game_2

Data Analysis Report

There will be **1 table** and **5 graphs** for different visualizations after the game ends (generated in game):

Table: Game summaries example:

Metric	Value
Total Turns	120
Soldiers Recruited	1,250
Soldiers Lost	980
Cities Gained	12
Cities Lost	8

Total Damage Dealt	5,430
Total Damage Taken	4,870
Game Result	Win

Graphs: Graph analysis criteria

Features	Objective	Type	X-Axis	Y-Axis
Soldiers Recruited vs Soldiers Lost	To give comparison on both subjects	Stacked Bar Chart	Turns	Number of soldiers
Damage Dealt vs Damage Taken	To give comparison on both subjects	Scatter Plot	Turns	Damage values (taken / dealt)
Cities Gained vs Cities Lost	To give comparison on both subjects	Bar Chart	Turns	Number of cities
Displays overall military growth across turns.	To show growth of powers in that game overtime	Cumulative Line Chart	Turns	Clan's power
Time per turn	To determine how much time player used per turn	Line Chart	Turns	Time

Developers Debugging

- The statistic will have debug data ready for displaying mock up data.
- This will trigger the gamestate to end and show the statistical mock-up data.

Document version

Version: 1.1

Date: 12 March 2026

Editor: Phunnipath Theankaew

Version: 1.2

Date: 21 March 2026

Editor: Phunnipath Theankaew

Version: 1.3

Date: 7 May 2026

Editor: Phunnipath Theankaew